genesis_sdlc User Guide

Version: 0.1.3 **GTL version**: 0.3.0

Contents

1. What genesis_sdlc Is
2. Installation
3. Your First Session
4. The Commands
5. The SDLC Graph
6. The Working Loop
7. Traceability
8. Writing Your Own Spec
9. Understanding the Self-Hosting Spec
10. Current Limitations

---

1. What genesis_sdlc Is

genesis_sdlc is an AI-augmented SDLC tool built on abiogenesis (the GTL engine). It provides a convergence-driven way to build software with AI: every stage has explicit acceptance criteria, progress is recorded in an append-only event log, and REQ keys thread from intent to runtime.

Teams using LLMs as code generators have no formal convergence criteria — no way to know when a stage is genuinely complete, no traceability from intent to runtime, no mechanism to detect drift. genesis_sdlc addresses this by defining the build as a typed directed graph and running the engine against it until every evaluator passes.

The SDLC graph

```
intent → requirements → feature_decomp → design → code ↔ unit_tests →
uat_tests
```

Seven asset types. Six edges. Each edge has evaluators that must pass before the graph advances.

The three evaluator kinds

Every evaluator belongs to one of three kinds:

Kind	Symbol	What it checks	Passes when
Deterministic test	`F_D`	Scripts, test suites, tag coverage — anything with a binary result	The command exits 0
Agent assessment	`F_P`	LLM judgment — does this output satisfy the spec?	An agent records a passing assessment in the event log
Human approval	`F_H`	Explicit sign-off at spec/design boundaries	A `review_approved` event exists for this edge

The engine runs deterministic tests first. Agent assessment only runs when all deterministic tests pass. Human approval only runs when agent assessment passes.

Delta and convergence

Each edge has a **delta** — the count of evaluators not yet passing. `delta = 0` means the edge is converged. The workspace is converged when every edge has `delta = 0`.

The event stream

All state lives in `.ai-workspace/events/events.jsonl`, an append-only log. Assets are not stored as mutable objects — they are derived by reading the event stream. Any past state is reconstructable by replaying the log.

---

2. Installation

Requirements

- Python 3.11 or later
- abiogenesis installed and on PATH (provides the `gen` CLI)
- A virtual environment is strongly recommended

Install into a target project

```
python -m genesis_sdlc.install --target /path/to/project
```

What the installer deploys:

Path	What it is
`.genesis/`	abiogenesis engine (via the abiogenesis installer)
`gtl_spec/packages/<slug>.py`	Starter SDLC spec — written once, never overwritten
`builds/python/`	`src/`, `tests/`, `design/adrs/` scaffold
`.claude/commands/gen-*.md`	All genesis slash commands
`CLAUDE.md`	Project orientation + Genesis Bootloader

Verify an existing install

```
python -m genesis_sdlc.install --target /path/to/project --verify
```

Reinstall (idempotent)

```
python -m genesis_sdlc.install --target /path/to/project
```

Engine files are always replaced. The starter spec (`gtl_spec/packages/<slug>.py`) is never replaced after the first install. The config file (`.genesis/genesis.yml`) is always replaced — it is engine metadata, not user data.

Platform scaffolding

```
# Default: Python
python -m genesis_sdlc.install --target . --project-slug myapp
```

```
# Explicit platform
python -m genesis_sdlc.install --target . --platform java --project-slug
        myapp
```

Running after install

```
cd /path/to/project
PYTHONPATH=.genesis python -m genesis gaps --workspace .
```

---

3. Your First Session

genesis_sdlc is self-hosting — the engine runs against its own workspace. The repo ships with a converged workspace, which makes it a good initial sanity check.

```
cd /path/to/genesis_sdlc
source .venv/bin/activate

# Check current state of the self-hosting workspace
PYTHONPATH=.genesis python -m genesis gaps --workspace .
```

Expected output (abbreviated):

```
{
  "scope": { "package": "genesis_sdlc", ... },
  "jobs_considered": 6,
  "total_delta": 0,
  "converged": true,
  "gaps": [ ... ]
}
```

`total_delta: 0` and `converged: true` mean every evaluator on every edge passes.

What `PYTHONPATH=.genesis` does

The self-hosting spec lives at `gtl_spec/packages/genesis_sdlc.py`. Python needs to find `gtl_spec` as a package. `PYTHONPATH=.genesis` adds the engine directory to the module search path so `from gtl_spec.packages.genesis_sdlc import ...` resolves correctly.

When genesis_sdlc is installed into another project, the spec lives inside the target project and PYTHONPATH is managed by the bootstrap contract (see §2).

---

4. The Commands

All commands accept:

- `--workspace DIR` — workspace root (default: current directory)
- `--package MODULE:VAR` — override the Package to load
- `--worker MODULE:VAR` — override the Worker to load

Without `--package`/`--worker`, commands read from `.genesis/genesis.yml`.

`gen gaps`

Computes residual work across the selected scope.

```
gen gaps --workspace .

# Scope to a specific feature
gen gaps --workspace . --feature REQ-F-GRAPH-001
```

Output fields:

Field	Meaning
`total_delta`	Sum of delta across all jobs. 0 = converged.
`converged`	`true` when `total_delta == 0`
`jobs_considered`	Number of jobs evaluated
`gaps[].edge`	Edge name
`gaps[].delta`	Residual for this edge
`gaps[].failing`	Evaluator names not yet passing
`gaps[].passing`	Evaluator names confirmed passing
`gaps[].delta_summary`	Human-readable summary line

When an edge reaches `delta = 0`, `gen gaps` emits an `edge_converged` event into the event log (idempotent — one certificate per edge).

`gen iterate`

Selects the first unconverged job and runs one bind-and-iterate pass.

```
gen iterate --workspace .

# Target a specific edge
gen iterate --workspace . --edge "design→code"
```

Output fields:

Field	Meaning
`status`	`iterated`, `converged`, or `nothing_to_do`
`edge`	Which edge was selected
`delta_before`	Delta prior to this iteration
`failing_evaluators`	Which evaluators were failing
`events_emitted`	How many events were written
`prompt_words`	Size of the F_P prompt (if dispatched)

What `gen iterate` does internally:

1. Runs all deterministic tests (`F_D`) and builds a manifest of what is passing and failing
2. If delta > 0, assembles an agent prompt with the relevant context documents
3. Walks evaluators in order: deterministic tests → agent assessment → human approval
4. Emits events from the working surface into the event log

`gen start`

State-machine wrapper over `gen iterate`. Without `--auto`, it behaves identically to `gen iterate`.

```
# Single iteration
gen start --workspace .
```

```
# Auto-loop: keep iterating until blocked
gen start --workspace . --auto
```

`--auto` stops when it encounters any condition requiring external input:

Stop reason	What happened
`converged`	All jobs delta = 0
`stopped_by: fp_dispatch`	Agent assessment dispatched — an agent needs to record results
`stopped_by: fh_gate`	Human approval gate is waiting
`stopped_by: fd_gap`	A deterministic test failed — artifacts need fixing
`stopped_by: max_iterations`	Safety limit of 50 iterations reached

`gen review`

Surfaces a candidate for human approval at an F_H gate.

```
gen review --feature REQ-F-GRAPH-001 --workspace .
```

After reviewing, approve by emitting a `review_approved` event:

```
{"event_type": "review_approved", "event_time": "...", "data": {"edge":
        "intent→requirements", "actor": "human"}}
```

Or via the engine CLI:

```
PYTHONPATH=.genesis python -m genesis emit-event \
    --type review_approved \
    --data '{"edge": "intent→requirements", "actor": "human"}'
```

`gen status`

Reports workspace state: recent events, active features, edge convergence summary.

```
gen status --workspace .
```

---

5. The SDLC Graph

The graph has seven assets and six edges.

Assets

Asset	id_format	Acceptance criteria (markov conditions)
`intent`	`INT-{SEQ}`	Problem stated, value proposition clear, scope bounded
`requirements`	`REQ-{SEQ}`	Keys testable, intent covered, no implementation details
`feature_decomp`	`FD-{SEQ}`	All REQ keys covered, dependency DAG acyclic, MVP boundary defined
`design`	`DES-{SEQ}`	ADRs recorded, tech stack decided, interfaces specified, no implementation details
`code`	`CODE-{SEQ}`	Implements tags present, importable, no V2 features

Asset	id_format	Acceptance criteria (markov conditions)
`unit_tests`	`TEST-{SEQ}`	All pass, validates tags present
`uat_tests`	`UAT-{SEQ}`	Sandbox install passes, e2e scenarios pass, accepted by human

Edges and evaluators

Edge	Evaluators	Kinds
`intent→requirements`	`intent_approved`	F_H
`requirements→feature_decomp`	`req_coverage`, `decomp_complete`, `decomp_approved`	F_D, F_P, F_H
`feature_decomp→design`	`design_coherent`, `design_approved`	F_P, F_H
`design→code`	`impl_tags`, `code_complete`	F_D, F_P
`code↔unit_tests`	`tests_pass`, `validates_tags`, `coverage_complete`	F_D, F_D, F_P
`unit_tests→uat_tests`	`uat_sandbox_report`, `uat_e2e_passed`, `uat_accepted`	F_D, F_P, F_H

Human gates

Three edges carry F_H evaluators:

- `intent→requirements` — confirms the problem is clearly stated before requirements work begins
- `requirements→feature_decomp` — confirms the feature set is complete and correctly ordered before design begins
- `feature_decomp→design` — confirms design before any code is written
- `unit_tests→uat_tests` — confirms sandbox e2e proof before shipping

The spec/design boundary sits at `feature_decomp→design`. Everything upstream is tech-agnostic (WHAT). Everything downstream is tech-bound (HOW).

The TDD edge

`code↔unit_tests` is a co-evolution edge — `co_evolve=True`. Code and tests are co-authored. The `code` asset is the source; `unit_tests` is the target. Both change together under the same evaluator set.

UAT constitutional requirement

Shipping requires sandbox e2e proof, not unit tests alone. The `unit_tests→uat_tests` edge enforces this. The F_D evaluator checks for a structured sandbox report at `.ai-workspace/uat/sandbox_report.json`. The F_P evaluator runs the full install into a fresh sandbox and writes that report. The F_H evaluator is the final sign-off.

---

6. The Working Loop

The standard cycle during active development:

```
# 1. See what is left
gen gaps --workspace .

# 2. Run one iteration (or auto-loop)
gen iterate --workspace .
# or: gen start --workspace . --auto

# 3. Read what happened
tail -5 .ai-workspace/events/events.jsonl

# 4. Respond to what the engine reported
#    fd_gap_found    → fix the deterministic failure, then return to step 1
#    fp_dispatched   → do the agent work, record the assessment, then return
       to step 1
#    fh_gate_pending → review and emit review_approved, then return to step 1

# 5. Confirm delta dropped
gen gaps --workspace .
```

The workspace is done when `gen gaps` reports `converged: true` and `total_delta: 0`.

Responding to each stop reason

`fd_gap_found` — a deterministic test failed. Read the failing evaluator name in the output, find the command in the spec (`gtl_spec/packages/genesis_sdlc.py`), run it manually to see the error, fix the artifact, and re-run.

`fp_dispatched` — an agent assessment evaluator fired. The engine emitted an `fp_dispatched` event and stopped. An agent (Claude, Codex) reads this event, does the work, and records the result. `gen iterate` does not invoke the agent directly — the agent invokes the engine. This separation keeps the engine deterministic.

`fh_gate_pending` — a human approval evaluator is waiting. Review the candidate artifact, then emit a `review_approved` event (see §4 `gen review`).

---

7. Traceability

REQ keys thread from the Package definition through code to tests.

How REQ keys flow

```
Package.requirements  →  feature vector satisfies:  →  # Implements: REQ-*  →
# Validates: REQ-*
```

The Package is the authoritative key registry. Feature vectors declare which keys they satisfy. Source files carry `# Implements:` tags. Test files carry `# Validates:` tags.

Check implementation tags

```
gen check-tags --type implements --path builds/python/src/
```

Output on success: `{"passes": true, "untagged_count": 0, "untagged": []}`. Exit code equals the untagged file count — 0 means all tagged.

Check test tags

```
gen check-tags --type validates --path builds/python/tests/
```

Check requirement coverage

```
gen check-req-coverage \
    --package gtl_spec.packages.genesis_sdlc:package \
    --features .ai-workspace/features/
```

Every REQ key in `Package.requirements` must appear in at least one feature vector's `satisfies:` field. This check is also wired as the `req_coverage` F_D evaluator on the `requirements→feature_decomp` edge — it runs automatically during `gen iterate`.

Feature vectors

Feature YAML files live in `.ai-workspace/features/active/` or `completed/`. A minimal vector:

```yaml
id: REQ-F-GRAPH-001
title: SDLC graph definition
status: active
satisfies:
  - REQ-F-GRAPH-001
  - REQ-F-GRAPH-002
```

Move from `active/` to `completed/` when the edge for that feature converges.

---

8. Writing Your Own Spec

genesis_sdlc installs a starter spec at `gtl_spec/packages/<slug>.py`. The starter spec is a copy of the SDLC bootstrap graph. Edit it to match your domain.

A spec exports two names: `package` and `worker`.

Minimal changes to the starter spec

1. **Rename assets** to match your domain artifacts
2. **Adjust markov conditions** — these are the acceptance criteria for each asset type
3. **Update evaluator commands** — `F_D` evaluators run subcommands; update paths to match your build layout
4. **Add or remove edges** — the graph is parameterisable; add intermediate nodes if the leap between two assets is too large for a single reliable construction step

Adding a deterministic evaluator

```
eval_schema = Evaluator(
    "schema_valid", F_D,
    "JSON schema validation passes",
    command="python -m pytest tests/test_schema.py -q",
)
```

The `command` field is the subprocess to run. Exit 0 = pass. The engine runs it as-is via `subprocess.run`.

Adding a human gate

```
standard_gate = Rule("standard_gate", approve=consensus(1, 1),
        dissent="recorded")

e_design_code = Edge(
    name="design→code",
    source=design,
    target=code,
    using=[claude_agent, human_gate],
    rule=standard_gate,
    context=[bootloader, this_spec],
)

eval_design_fh = Evaluator(
    "design_approved", F_H,
    "Human approves design before any code is written",
)
```

The F_H evaluator passes when a `review_approved` event exists in the event log for this edge.

Updating the Package requirements list

```
package = Package(
    name="my_domain",
    ...
    requirements=[
        "REQ-F-AUTH-001",
        "REQ-F-AUTH-002",
        "REQ-F-DATA-001",
    ],
)
```

`check-req-coverage` uses this list as the authoritative registry. Add keys here as requirements are written.

---

9. Understanding the Self-Hosting Spec

`gtl_spec/packages/genesis_sdlc.py` is the V1 specification written as GTL. It defines genesis_sdlc itself.

The spec is also the template installed into target projects. When a team installs genesis_sdlc, they receive a copy of this spec as their starting point. The engine that checks their project is the same engine that was used to build genesis_sdlc.

The self-hosting workspace

`.ai-workspace/events/events.jsonl` contains the build certificates for genesis_sdlc itself. The five major edges are certified converged. `gen gaps --workspace .` on the genesis_sdlc repo returns `converged: true`.

The context surfaces

Each edge loads specific context documents into the agent prompt:

Context	Locator	Loaded on
`bootloader`	`gtl_spec/GENESIS_BOOTLOADER.md`	All edges
`genesis_sdlc_spec`	`gtl_spec/packages/genesis_sdlc.py`	All edges
`intent`	`INTENT.md`	requirements, feature_decomp, design edges
`design_adrs`	`builds/python/design/adrs/`	design, code, TDD, UAT edges

The bootloader carries the GTL constraint axioms. The spec carries the graph definition. Together they give the agent enough context to construct and evaluate work on any edge without reading additional documents.

---

10. Current Limitations

V1 scoping

`--feature REQ-F-GRAPH-001` validates that the feature ID exists in `.ai-workspace/features/` but does not narrow which jobs run. All jobs run regardless of feature scope. Per-job feature routing is V2.

Agent dispatch is asynchronous

When an F_P evaluator fires, `gen iterate` emits an `fp_dispatched` event and stops. The agent reads this event, does the work, and records the result externally. The engine does not invoke the agent directly — the agent invokes the engine. This keeps the engine deterministic.

Human approval requires a manual event

To clear an F_H gate, emit a `review_approved` event into the event log:

```
{"event_type": "review_approved", "event_time": "...", "data": {"edge":
        "intent→requirements", "actor": "human"}}
```

Once this event exists, the F_H evaluator passes on the next `gen gaps` or `gen iterate` call.

F_D evaluators must be acyclic

Deterministic test commands must never invoke genesis subcommands. Calling `gen gaps` from inside a pytest test creates unbounded subprocess recursion. The `code↔unit_tests` edge runs pytest with `-m 'not e2e'` to exclude tests that call genesis. Apply the same pattern in your own spec.

Single worker

V1 has one worker (`claude_code`). Multi-tenant scheduling with conflict detection is deferred to V2. The `schedule()` function and `Worker.conflicts_with()` are implemented but only exercised with a single worker in V1.

UAT edge

The `unit_tests→uat_tests` edge is defined in the spec and the evaluators are wired. The F_D sandbox report check and F_P sandbox install evaluator are complete. The edge is in active development and is not yet certified converged in the self-hosting workspace.

Source layout

Engine source is at `builds/python/src/genesis_sdlc/` — not `src/genesis_sdlc/`. The `builds/` prefix reflects the self-hosting structure: the engine is itself a build artifact described by the spec. A conventional `src/` layout is deferred until the packaging migration is complete.